```
import pandas as pd
```

```
import numpy as np
```

```
df = pd.read_csv('/content/creditcard.csv')
```

```
print(df)
```

```
        Time        V1        V2        V3        V4        V5        V6  \
0          0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388
1          0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361
2          1 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499
3          1 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203
4          2 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921
...      ...       ...       ...       ...       ...       ...       ...
91195  63353 -0.490895  1.107765  1.502218  0.848157  0.424492 -0.116355
91196  63353  1.263123 -0.735270 -0.001407 -0.796592 -0.759490 -0.369354
91197  63354 -0.223639  1.073431  1.331846  0.463195  0.135678 -0.881074
91198  63354  1.135869 -1.278531  0.076387 -0.002852  0.679808  4.766929
91199  63354 -5.923850  5.078018 -3.223655  1.053491 -3.230265 -0.405473

             V7        V8        V9  ...       V21       V22       V23  \
0      0.239599  0.098698  0.363787  ... -0.018307  0.277838 -0.110474
1     -0.078803  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288
2      0.791461  0.247676 -1.514654  ...  0.247998  0.771679  0.909412
3      0.237609  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321
4      0.592941 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458
...         ...       ...       ...  ...       ...       ...       ...
91195  0.801624  0.093389 -1.189178  ...  0.145558  0.422849 -0.246593
91196 -0.406820 -0.057882 -1.063382  ... -0.545363 -1.335962  0.035395
91197  0.708761 -0.181839 -0.726279  ... -0.143288 -0.387236  0.056717
91198 -1.836080  1.259028  0.439912  ... -0.508605 -0.798310 -0.081020
91199 -2.966812  4.288420  0.272339  ...       NaN       NaN       NaN

            V24       V25       V26       V27       V28  Amount  Class
0      0.066928  0.128539 -0.189115  0.133558 -0.021053  149.62    0.0
1     -0.339846  0.167170  0.125895 -0.008983  0.014724    2.69    0.0
2     -0.689281 -0.327642 -0.139097 -0.055353 -0.059752  378.66    0.0
3     -1.175575  0.647376 -0.221929  0.062723  0.061458  123.50    0.0
4      0.141267 -0.206010  0.502292  0.219422  0.215153   69.99    0.0
...         ...       ...       ...       ...       ...     ...    ...
91195  0.229489  0.107284 -0.263054  0.110716  0.094024    4.99    0.0
91196 -0.359533  0.081162  0.835179 -0.081592  0.004941   79.38    0.0
91197  0.371521 -0.702896  0.084805  0.140208  0.177478    6.99    0.0
91198  0.996750  0.571672 -0.268628  0.118991  0.046959   75.00    0.0
91199       NaN       NaN       NaN       NaN       NaN     NaN    NaN

[91200 rows x 31 columns]
```

```
print(df.columns)
```

```
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
       'Class'],
      dtype='object')
```

```
# 1. Dataset shape
total_rows, total_columns = df.shape
```

```
print(df.shape)
```

```
(91200, 31)
```

```
# 2. Missing values
total_missing = int(df.isnull().sum().sum())
missing_by_column = df.isnull().sum()
columns_with_missing = missing_by_column[missing_by_column > 0]
```

```
import pandas as pd
import numpy as np

# Load the Kaggle dataset
```

```
df = pd.read_csv("/content/creditcard.csv")

# 1. Dataset shape
total_rows, total_columns = df.shape

# 2. Missing values
total_missing = int(df.isnull().sum().sum())
missing_by_column = df.isnull().sum()
columns_with_missing = missing_by_column[missing_by_column > 0]

# 3. Duplicate rows
duplicate_rows = int(df.duplicated().sum())

# 4. Fraud and legitimate transactions
fraud_count = int((df["Class"] == 1).sum())
legitimate_count = int((df["Class"] == 0).sum())

# 5. Fraud percentage
fraud_percentage = (fraud_count / total_rows) * 100
legitimate_percentage = (legitimate_count / total_rows) * 100

# Display results
print("=" * 45)
print("FRAUDSHIELD AI - DATASET AUDIT")
print("=" * 45)

print(f"Dataset rows             : {total_rows:,}")
print(f"Dataset columns          : {total_columns}")
print(f"Total missing values     : {total_missing:,}")
print(f"Duplicate rows           : {duplicate_rows:,}")
print(f"Legitimate transactions  : {legitimate_count:,}")
print(f"Fraudulent transactions  : {fraud_count:,}")
print(f"Fraud percentage         : {fraud_percentage:.4f}%")
print(f"Legitimate percentage    : {legitimate_percentage:.4f}%")

print("\nMissing Values by Column:")
if columns_with_missing.empty:
    print("No missing values found.")
else:
    print(columns_with_missing.sort_values(ascending=False))

# Class distribution table
class_summary = (
    df["Class"]
    .value_counts()
    .rename(index={0: "Legitimate", 1: "Fraud"})
    .to_frame("Transaction Count")
)

class_summary["Percentage"] = (
    class_summary["Transaction Count"] / total_rows * 100
).round(4)

print("\nClass Distribution:")
print(class_summary)
```

```
=============================================
FRAUDSHIELD AI - DATASET AUDIT
=============================================
Dataset rows             : 284,807
Dataset columns          : 31
Total missing values     : 0
Duplicate rows           : 1,081
Legitimate transactions  : 284,315
Fraudulent transactions  : 492
Fraud percentage         : 0.1727%
Legitimate percentage    : 99.8273%

Missing Values by Column:
No missing values found.

Class Distribution:
            Transaction Count  Percentage
Class
Legitimate             284315     99.8273
Fraud                     492      0.1727
```

```
# FraudShield AI - Duplicate Investigation

# Identify all rows that belong to duplicate groups
duplicate_mask = df.duplicated(keep=False)

duplicate_records = df.loc[duplicate_mask].copy()

print("DUPLICATE INVESTIGATION")
print("-" * 40)

print("Rows marked as duplicates (excluding first occurrences):",
      df.duplicated().sum())

print("Rows belonging to duplicate groups:",
      duplicate_mask.sum())

print("\nClass distribution among duplicate-group rows:")
print(
    duplicate_records["Class"]
    .value_counts()
    .rename(index={0: "Legitimate", 1: "Fraud"})
)

print("\nFraud rate among duplicate-group rows:")
print(
    (duplicate_records["Class"].mean() * 100).round(4),
    "%"
)

print("\nExact duplicate rows by Class:")
print(
    df.loc[df.duplicated(keep="first"), "Class"]
    .value_counts()
    .rename(index={0: "Legitimate", 1: "Fraud"})
)

print("\nFirst 10 duplicate-group records:")
display(duplicate_records.head(10))
```

```
DUPLICATE INVESTIGATION
----------------------------------------
Rows marked as duplicates (excluding first occurrences): 1081
Rows belonging to duplicate groups: 1854

Class distribution among duplicate-group rows:
Class
Legitimate    1822
Fraud           32
Name: count, dtype: int64

Fraud rate among duplicate-group rows:
1.726 %

Exact duplicate rows by Class:
Class
Legitimate    1062
Fraud           19
Name: count, dtype: int64

First 10 duplicate-group records:
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	
32	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206	0.176066	-0.286717	...	0.046949	0.208105	-0.18
33	26.0	-0.529912	0.873892	1.347247	0.145457	0.414209	0.100223	0.711206	0.176066	-0.286717	...	0.046949	0.208105	-0.18
34	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039	0.179742	-0.285642	...	0.049526	0.206537	-0.18
35	26.0	-0.535388	0.865268	1.351076	0.147575	0.433680	0.086983	0.693039	0.179742	-0.285642	...	0.049526	0.206537	-0.18
112	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715	0.350995	0.118950	...	0.102520	0.605089	0.02
113	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715	0.350995	0.118950	...	0.102520	0.605089	0.02
114	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715	0.350995	0.118950	...	0.102520	0.605089	0.02
115	74.0	1.038370	0.127486	0.184456	1.109950	0.441699	0.945283	-0.036715	0.350995	0.118950	...	0.102520	0.605089	0.02
220	145.0	-2.420413	1.947885	0.553646	0.983069	-0.281518	2.408958	-1.401613	-0.188299	0.675878	...	1.213826	-1.238620	0.00
221	145.0	-2.420413	1.947885	0.553646	0.983069	-0.281518	2.408958	-1.401613	-0.188299	0.675878	...	1.213826	-1.238620	0.00

10 rows × 31 columns

```
# FraudShield AI - Compare Original vs Deduplicated Data

# Preserve the raw dataset
df_raw = df.copy()

# Create a separate version without exact duplicate rows
df_clean = df_raw.drop_duplicates().copy()

def fraud_summary(data, dataset_name):
    total = len(data)
    fraud = int((data["Class"] == 1).sum())
    legitimate = int((data["Class"] == 0).sum())

    return {
        "Dataset": dataset_name,
        "Rows": total,
        "Removed vs Original": len(df_raw) - total,
        "Fraud Cases": fraud,
        "Legitimate Cases": legitimate,
        "Fraud Rate (%)": round(fraud / total * 100, 4)
    }

comparison = pd.DataFrame([
    fraud_summary(df_raw, "Original"),
    fraud_summary(df_clean, "Deduplicated")
])

display(comparison)

# Check how many fraud and legitimate labels were removed
removed_rows = df_raw.loc[
    ~df_raw.index.isin(df_clean.index)
]
```

```
print("\nRemoved duplicate occurrences by class:")
print(
    removed_rows["Class"]
    .value_counts()
    .rename(index={0: "Legitimate", 1: "Fraud"})
)
```

	Dataset	Rows	Removed vs Original	Fraud Cases	Legitimate Cases	Fraud Rate (%)
0	Original	284807	0	492	284315	0.1727
1	Deduplicated	283726	1081	473	283253	0.1667

```
Removed duplicate occurrences by class:
Class
Legitimate    1062
Fraud           19
Name: count, dtype: int64
```

Next steps: Generate code with comparison | New interactive sheet

```
from pathlib import Path

# Preserve the original dataset separately
df_raw.to_csv("creditcard_original.csv", index=False)

# Save the deduplicated working dataset
df_clean.to_csv("creditcard_cleaned.csv", index=False)

print("Original dataset:", df_raw.shape)
print("Cleaned dataset:", df_clean.shape)
print("Cleaned file saved:", Path("creditcard_cleaned.csv").resolve())
```

```
Original dataset: (284807, 31)
Cleaned dataset: (283726, 31)
Cleaned file saved: /content/creditcard_cleaned.csv
```

```
# FraudShield AI - Transaction Amount Analysis

amount_summary = (
    df_clean.groupby("Class")["Amount"]
    .agg(["count", "mean", "median", "std", "min", "max"])
    .rename(index={
        0: "Legitimate",
        1: "Fraud"
    })
    .round(2)
)

display(amount_summary)

# Compare transaction amount percentiles
percentiles = (
    df_clean.groupby("Class")["Amount"]
    .quantile([0.25, 0.50, 0.75, 0.90, 0.95, 0.99])
    .unstack()
    .rename(index={
        0: "Legitimate",
        1: "Fraud"
    })
    .rename(columns={
        0.25: "25th percentile",
        0.50: "50th percentile",
        0.75: "75th percentile",
        0.90: "90th percentile",
        0.95: "95th percentile",
        0.99: "99th percentile"
    })
    .round(2)
)
```

```
display(percentiles)
```

Class	count	mean	median	std	min	max
Legitimate	283253	88.41	22.00	250.38	0.0	25691.16
Fraud	473	123.87	9.82	260.21	0.0	2125.87

Class	25th percentile	50th percentile	75th percentile	90th percentile	95th percentile	99th percentile
Legitimate	5.67	22.00	77.46	203.00	365.00	1018.06
Fraud	1.00	9.82	105.89	348.65	655.82	1364.14

Next steps: Generate code with amount_summary | New interactive sheet | Generate code with percentiles | New interactive sheet

```
# FraudShield AI - Fraud Rate by Transaction Amount Range

import pandas as pd
import numpy as np

amount_bins = [
    -0.01, 10, 50, 100, 250, 500, 1000, float("inf")
]

amount_labels = [
    "0–10",
    "10–50",
    "50–100",
    "100–250",
    "250–500",
    "500–1000",
    "1000+"
]

df_amount = df_clean.copy()

df_amount["Amount_Range"] = pd.cut(
    df_amount["Amount"],
    bins=amount_bins,
    labels=amount_labels,
    include_lowest=True
)

amount_risk = (
    df_amount.groupby("Amount_Range", observed=False)
    .agg(
        Total_Transactions=("Class", "size"),
        Fraud_Transactions=("Class", "sum"),
        Average_Amount=("Amount", "mean")
    )
)

amount_risk["Legitimate_Transactions"] = (
    amount_risk["Total_Transactions"]
    - amount_risk["Fraud_Transactions"]
)

amount_risk["Fraud_Rate_Pct"] = (
    amount_risk["Fraud_Transactions"]
    / amount_risk["Total_Transactions"]
    * 100
)

amount_risk["Share_of_All_Fraud_Pct"] = (
    amount_risk["Fraud_Transactions"]
    / df_amount["Class"].sum()
    * 100
)

amount_risk = amount_risk.round(2)

display(amount_risk)
```

Amount_Range	Total_Transactions	Fraud_Transactions	Average_Amount	Legitimate_Transactions	Fraud_Rate_Pct	Share_of_All_Frau
0–10	99821	238	3.89	99583	0.24	
10–50	90327	55	25.97	90272	0.06	
50–100	37179	55	73.31	37124	0.15	
100–250	33828	54	158.15	33774	0.16	
250–500	13462	37	349.42	13425	0.27	
500–1000	6174	25	693.41	6149	0.40	
1000+	2935	9	1808.40	2926	0.31	

Next steps: Generate code with `amount_risk` New interactive sheet

```
# FraudShield AI - Fraud Patterns Over Elapsed Time

import pandas as pd
import numpy as np

df_time = df_clean.copy()

# Convert elapsed seconds into elapsed hours
df_time["Elapsed_Hours"] = df_time["Time"] / 3600

# Create 6-hour intervals
max_hour = int(np.ceil(df_time["Elapsed_Hours"].max()))

time_bins = list(range(0, max_hour + 6, 6))
time_bins[0] = -0.001

time_labels = [
    f"{int(time_bins[i])}–{int(time_bins[i + 1])} hours"
    for i in range(len(time_bins) - 1)
]

df_time["Elapsed_Time_Range"] = pd.cut(
    df_time["Elapsed_Hours"],
    bins=time_bins,
    labels=time_labels,
    right=False
)

time_risk = (
    df_time.groupby("Elapsed_Time_Range", observed=False)
    .agg(
        Total_Transactions=("Class", "size"),
        Fraud_Transactions=("Class", "sum")
    )
)

time_risk["Legitimate_Transactions"] = (
    time_risk["Total_Transactions"]
    - time_risk["Fraud_Transactions"]
)

time_risk["Fraud_Rate_Pct"] = (
    time_risk["Fraud_Transactions"]
    / time_risk["Total_Transactions"].replace(0, np.nan)
    * 100
)

time_risk["Share_of_All_Fraud_Pct"] = (
    time_risk["Fraud_Transactions"]
    / df_time["Class"].sum()
    * 100
)

display(time_risk.round(3))
```

Elapsed_Time_Range	Total_Transactions	Fraud_Transactions	Legitimate_Transactions	Fraud_Rate_Pct	Share_of_All_Fraud_Pct
0–6 hours	12296	55	12241	0.447	11.628
6–12 hours	34897	91	34806	0.261	19.239
12–18 hours	46731	71	46660	0.152	15.011
18–24 hours	50312	55	50257	0.109	11.628
24–30 hours	11546	60	11486	0.520	12.685
30–36 hours	35746	27	35719	0.076	5.708
36–42 hours	49390	62	49328	0.126	13.108
42–48 hours	42808	52	42756	0.121	10.994

```
# FraudShield AI - Anonymized Feature Analysis

import numpy as np
import pandas as pd
```

```
feature_cols = [f"V{i}" for i in range(1, 29)]

legit = df_clean.loc[df_clean["Class"] == 0, feature_cols]
fraud = df_clean.loc[df_clean["Class"] == 1, feature_cols]

feature_comparison = pd.DataFrame({
    "Legitimate_Mean": legit.mean(),
    "Fraud_Mean": fraud.mean(),
    "Legitimate_Median": legit.median(),
    "Fraud_Median": fraud.median(),
    "Legitimate_Std": legit.std(),
    "Fraud_Std": fraud.std()
})

# Standardized mean difference using pooled standard deviation
pooled_std = np.sqrt(
    (
        (len(legit) - 1) * legit.var()
        + (len(fraud) - 1) * fraud.var()
    )
    / (len(legit) + len(fraud) - 2)
)

feature_comparison["Standardized_Mean_Difference"] = (
    (fraud.mean() - legit.mean()) / pooled_std
)

feature_comparison["Absolute_Difference_Score"] = (
    feature_comparison["Standardized_Mean_Difference"].abs()
)

feature_comparison = feature_comparison.sort_values(
    "Absolute_Difference_Score",
    ascending=False
).round(4)

display(feature_comparison.head(10))
```

	Legitimate_Mean	Fraud_Mean	Legitimate_Median	Fraud_Median	Legitimate_Std	Fraud_Std	Standardized_Mean_Difference	Abso
V17	0.0110	-6.4633	-0.0651	-5.1576	0.7485	6.9657	-8.0924	
V14	0.0117	-6.8359	0.0515	-6.5906	0.8944	4.2532	-7.5222	
V12	0.0095	-6.1033	0.1406	-5.4374	0.9454	4.5823	-6.3482	
V10	0.0077	-5.4533	-0.0921	-4.4663	1.0363	4.7065	-5.1856	
V16	0.0078	-4.0010	0.0681	-3.3029	0.8446	3.8317	-4.6709	
V3	0.0129	-6.7296	0.1822	-4.8754	1.4576	6.9096	-4.5453	
V7	0.0104	-5.1759	0.0417	-2.9021	1.1775	6.8580	-4.2887	
V11	-0.0060	3.7163	-0.0344	3.5257	1.0023	2.6728	3.6952	
V4	-0.0104	4.4726	-0.0245	4.1001	1.3986	2.8715	3.1969	
V18	0.0051	-2.1571	-0.0014	-1.4179	0.8250	2.9018	-2.5965	

```
# FraudShield AI - Top Fraud-Associated Feature Distributions

import matplotlib.pyplot as plt
import seaborn as sns

top_features = ["V17", "V14", "V12", "V10", "V16", "V3"]

plot_data = df_clean[["Class"] + top_features].copy()
plot_data["Transaction_Type"] = plot_data["Class"].map({
    0: "Legitimate",
    1: "Fraud"
})

fig, axes = plt.subplots(2, 3, figsize=(16, 9))

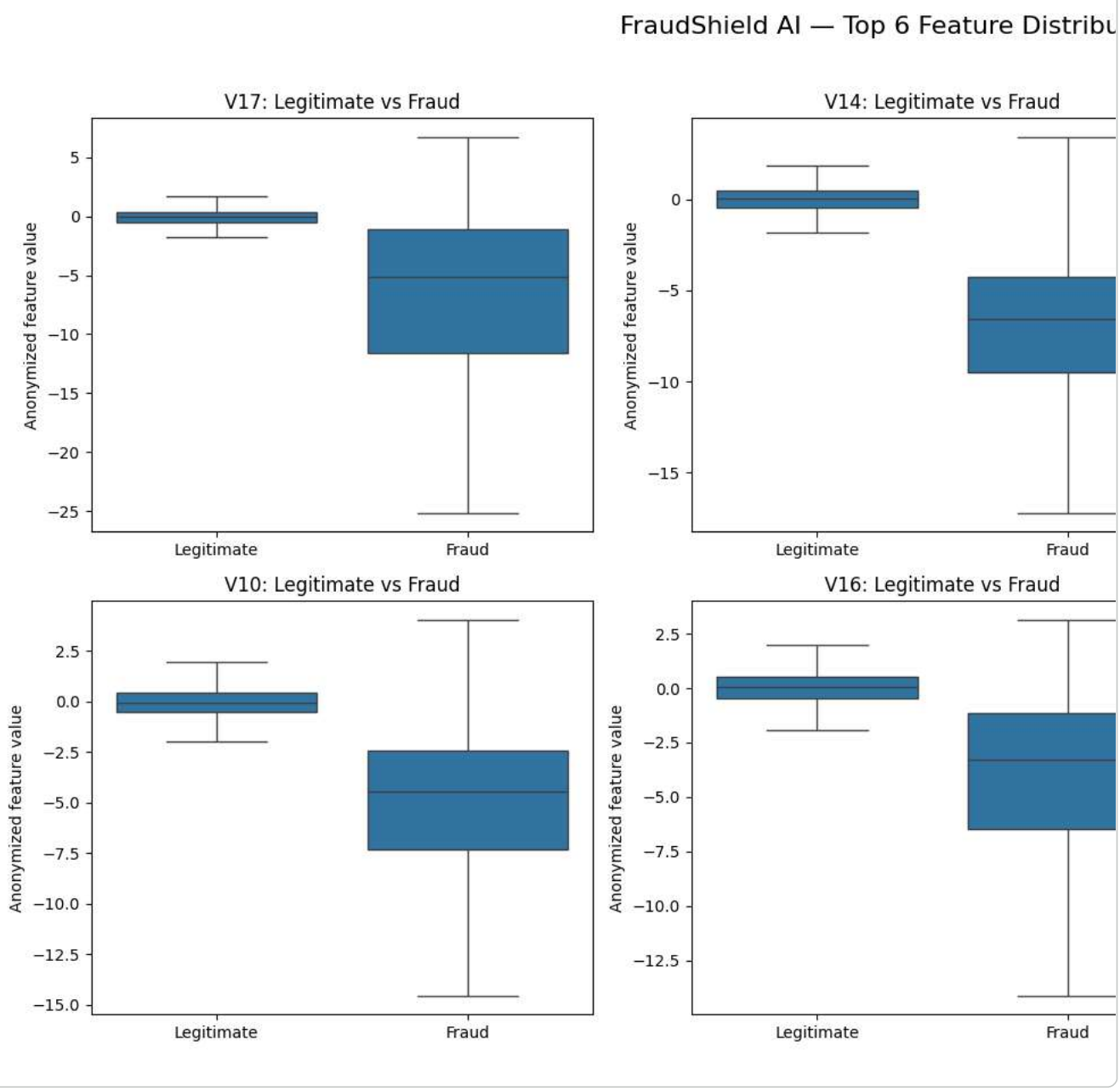
for feature, ax in zip(top_features, axes.flatten()):
    sns.boxplot(
        data=plot_data,
        x="Transaction_Type"
```

```
        x="Transaction_Type",
        y=feature,
        order=["Legitimate", "Fraud"],
        showfliers=False,
        ax=ax
    )

    ax.set_title(f"{feature}: Legitimate vs Fraud")
    ax.set_xlabel("")
    ax.set_ylabel("Anonymized feature value")

plt.suptitle(
    "FraudShield AI — Top 6 Feature Distributions",
    fontsize=16,
    y=1.02
)

plt.tight_layout()
plt.show()
```




```
# FraudShield AI - Final EDA Validation

import pandas as pd
import numpy as np

print("DATASET VALIDATION")
```

```
print("-" * 45)
print("Rows:", len(df_clean))
print("Columns:", df_clean.shape[1])
print("Missing values:", df_clean.isna().sum().sum())
print("Duplicate rows remaining:", df_clean.duplicated().sum())

print("\nCLASS DISTRIBUTION")
class_summary = (
    df_clean["Class"]
    .value_counts()
    .rename(index={0: "Legitimate", 1: "Fraud"})
    .to_frame("Transactions")
)

class_summary["Percentage"] = (
    class_summary["Transactions"] / len(df_clean) * 100
).round(4)

display(class_summary)

assert set(df_clean["Class"].unique()).issubset({0, 1})
assert df_clean["Class"].nunique() == 2
assert df_clean.isna().sum().sum() == 0
assert df_clean.duplicated().sum() == 0
```

```
DATASET VALIDATION
--------------------------------------------
Rows: 283726
Columns: 31
Missing values: 0
Duplicate rows remaining: 0

CLASS DISTRIBUTION
```

Class	Transactions	Percentage
Legitimate	283253	99.8333
Fraud	473	0.1667

Next steps: Generate code with `class_summary` | New interactive sheet

```
# FraudShield AI - Feature Correlation with Fraud Label

feature_cols = [f"V{i}" for i in range(1, 29)]

correlation_with_fraud = (
    df_clean[feature_cols + ["Amount", "Time", "Class"]]
    .corr(numeric_only=True)["Class"]
    .drop("Class")
    .to_frame("Correlation_with_Class")
)

correlation_with_fraud["Absolute_Correlation"] = (
    correlation_with_fraud["Correlation_with_Class"].abs()
)

correlation_with_fraud = correlation_with_fraud.sort_values(
    "Absolute_Correlation",
    ascending=False
).round(4)

display(correlation_with_fraud.head(15))
```

	Correlation_with_Class	Absolute_Correlation
V17	-0.3135	0.3135
V14	-0.2934	0.2934
V12	-0.2507	0.2507
V10	-0.2070	0.2070
V16	-0.1872	0.1872
V3	-0.1823	0.1823
V7	-0.1723	0.1723
V11	0.1491	0.1491
V4	0.1293	0.1293
V18	-0.1053	0.1053
V1	-0.0945	0.0945
V9	-0.0940	0.0940
V5	-0.0878	0.0878
V2	0.0846	0.0846
V6	-0.0439	0.0439

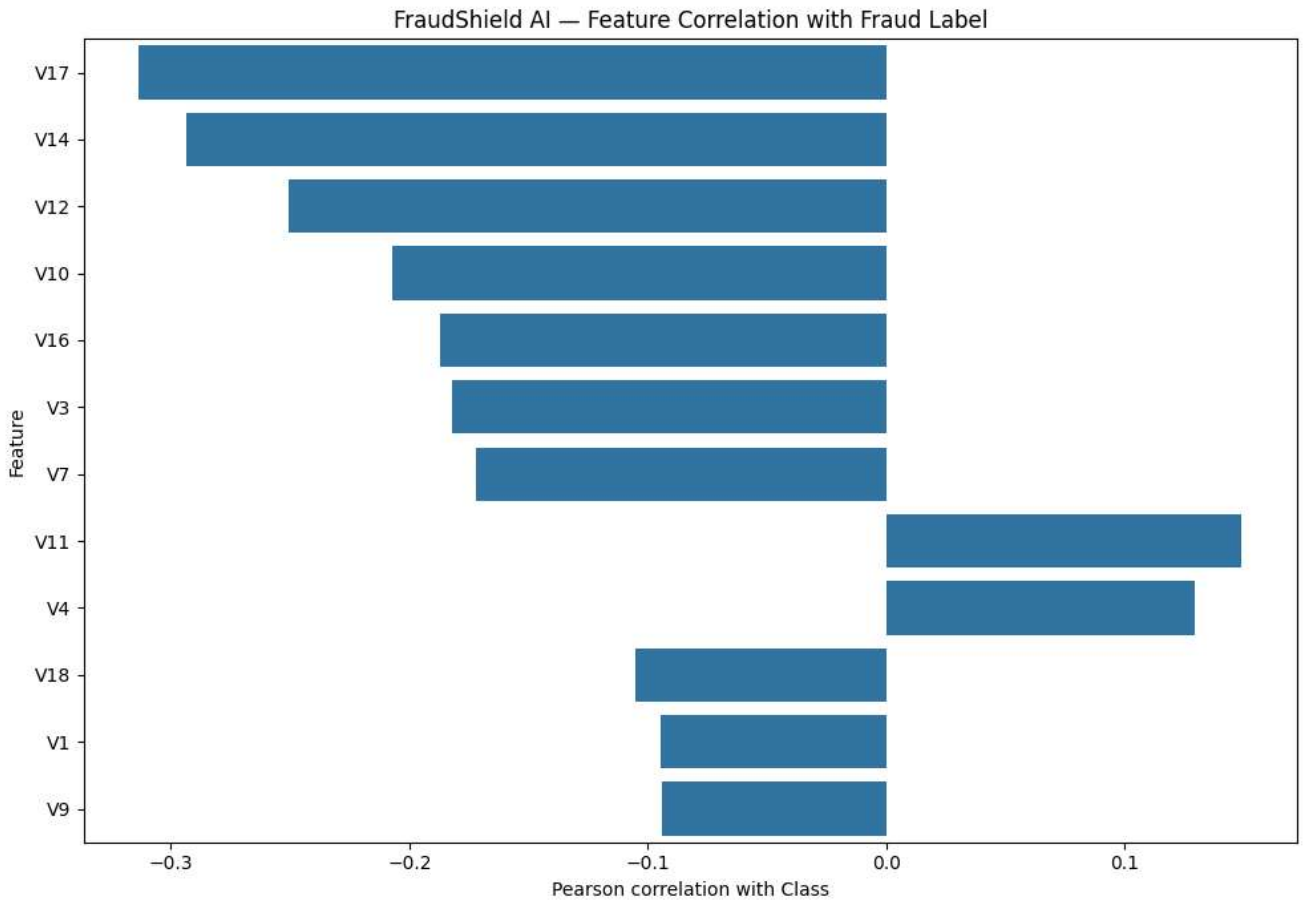
```
import matplotlib.pyplot as plt
import seaborn as sns

top_corr_features = correlation_with_fraud.head(12).index.tolist()

plt.figure(figsize=(10, 7))

sns.barplot(
    data=correlation_with_fraud.loc[top_corr_features].reset_index(),
    x="Correlation_with_Class",
    y="index"
)

plt.title("FraudShield AI — Feature Correlation with Fraud Label")
plt.xlabel("Pearson correlation with Class")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()
```




```
# FraudShield AI - Baseline Logistic Regression Model

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    average_precision_score,
    roc_auc_score
)

# 1. Separate features and target
X = df_clean.drop(columns=["Class"])
y = df_clean["Class"]

# 2. Stratified train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

print("TRAINING DATA")
print("Training rows:", len(X_train))
print("Training fraud cases:", int(y_train.sum()))

print("\nTEST DATA")
print("Testing rows:", len(X_test))
print("Testing fraud cases:", int(y_test.sum()))

# 3. Create pipeline
fraud_model = Pipeline([
```

```
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        class_weight="balanced",
        max_iter=2000,
        random_state=42
    ))
])

# 4. Train the model
fraud_model.fit(X_train, y_train)

# 5. Predict on unseen test data
y_pred = fraud_model.predict(X_test)
y_scores = fraud_model.predict_proba(X_test)[:, 1]

# 6. Evaluation metrics
print("\nCLASSIFICATION REPORT")
print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1],
    target_names=["Legitimate", "Fraud"],
    digits=4,
    zero_division=0
))

print("CONFUSION MATRIX")
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

display(pd.DataFrame(
    cm,
    index=["Actual Legitimate", "Actual Fraud"],
    columns=["Predicted Legitimate", "Predicted Fraud"]
))

print("\nOVERALL MODEL METRICS")
print(f"Average Precision: {average_precision_score(y_test, y_scores):.4f}")
print(f"ROC-AUC:           {roc_auc_score(y_test, y_scores):.4f}")
```

```
TRAINING DATA
Training rows: 226980
Training fraud cases: 378

TEST DATA
Testing rows: 56746
Testing fraud cases: 95

CLASSIFICATION REPORT
              precision    recall  f1-score   support

  Legitimate     0.9998    0.9755    0.9875     56651
       Fraud     0.0564    0.8737    0.1059        95

    accuracy                         0.9753     56746
   macro avg     0.5281    0.9246    0.5467     56746
weighted avg     0.9982    0.9753    0.9860     56746

CONFUSION MATRIX
```

	Predicted Legitimate	Predicted Fraud
Actual Legitimate	55262	1389
Actual Fraud	12	83

```
OVERALL MODEL METRICS
Average Precision: 0.6719
ROC-AUC:           0.9657
```

```
# FraudShield AI - Validation-Based Alert Threshold Analysis

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
```

```
# Split the existing training data into model-fit and validation sets
X_fit, X_val, y_fit, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.25,
    stratify=y_train,
    random_state=42
)

# Train a fresh model using only the model-fit subset
validation_model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(
        class_weight="balanced",
        max_iter=2000,
        random_state=42
    ))
])

validation_model.fit(X_fit, y_fit)

# Obtain fraud probabilities for the validation set
val_scores = validation_model.predict_proba(X_val)[:, 1]

# Compare alert thresholds
thresholds = [0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.70, 0.80, 0.90]

results = []

for threshold in thresholds:
    val_pred = (val_scores >= threshold).astype(int)

    tp = int(((y_val == 1) & (val_pred == 1)).sum())
    fp = int(((y_val == 0) & (val_pred == 1)).sum())
    fn = int(((y_val == 1) & (val_pred == 0)).sum())

    results.append({
        "Threshold": threshold,
        "Fraud_Caught": tp,
        "Fraud_Missed": fn,
        "False_Alerts": fp,
        "Total_Alerts": tp + fp,
        "Precision_Pct": precision_score(
            y_val, val_pred, zero_division=0
        ) * 100,
        "Recall_Pct": recall_score(
            y_val, val_pred, zero_division=0
        ) * 100,
        "F1_Score": f1_score(
            y_val, val_pred, zero_division=0
        )
    })

threshold_results = pd.DataFrame(results)

display(threshold_results.round(3))
```

	Threshold	Fraud_Caught	Fraud_Missed	False_Alerts	Total_Alerts	Precision_Pct	Recall_Pct	F1_Score
0	0.1	90	4	9348	9438	0.954	95.745	0.019
1	0.2	88	6	4818	4906	1.794	93.617	0.035
2	0.3	86	8	3070	3156	2.725	91.489	0.053
3	0.4	86	8	2095	2181	3.943	91.489	0.076
4	0.5	85	9	1495	1580	5.380	90.426	0.102
5	0.6	85	9	1021	1106	7.685	90.426	0.142
6	0.7	85	9	743	828	10.266	90.426	0.184
7	0.8	84	10	494	578	14.533	89.362	0.250
8	0.9	83	11	254	337	24.629	88.298	0.385

```
# FraudShield AI - Final Threshold Comparison on Test Set
```

```
import pandas as pd
from sklearn.base import clone
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

# Retrain on the full training split
final_model = clone(fraud_model)
final_model.fit(X_train, y_train)

# Get fraud probabilities for the held-out test set
test_scores = final_model.predict_proba(X_test)[:, 1]

threshold_results_test = []

for threshold in [0.5, 0.8, 0.9]:
    predictions = (test_scores >= threshold).astype(int)

    tn, fp, fn, tp = confusion_matrix(
        y_test, predictions, labels=[0, 1]
    ).ravel()

    threshold_results_test.append({
        "Threshold": threshold,
        "True_Positives": tp,
        "Fraud_Missed": fn,
        "False_Alerts": fp,
        "True_Negatives": tn,
        "Total_Alerts": tp + fp,
        "Precision_Pct": precision_score(
            y_test, predictions, zero_division=0
        ) * 100,
        "Recall_Pct": recall_score(
            y_test, predictions, zero_division=0
        ) * 100,
        "F1_Score": f1_score(
            y_test, predictions, zero_division=0
        )
    })

test_threshold_comparison = pd.DataFrame(
    threshold_results_test
)

display(test_threshold_comparison.round(3))
```

	Threshold	True_Positives	Fraud_Missed	False_Alerts	True_Negatives	Total_Alerts	Precision_Pct	Recall_Pct	F1_Score
0	0.5	83	12	1389	55262	1472	5.639	87.368	0.106
1	0.8	80	15	419	56232	499	16.032	84.211	0.269
2	0.9	79	16	222	56429	301	26.246	83.158	0.399

```
# FraudShield AI - Export Model Results for Power BI

from pathlib import Path
import pandas as pd
import numpy as np
from sklearn.metrics import confusion_matrix

output_dir = Path("fraudshield_powerbi")
output_dir.mkdir(exist_ok=True)

# Candidate threshold chosen from validation analysis
selected_threshold = 0.8

# Transaction-level prediction output
powerbi_predictions = df_clean.loc[X_test.index].copy()

powerbi_predictions["Actual_Class"] = y_test
powerbi_predictions["Fraud_Probability"] = test_scores
powerbi_predictions["Selected_Threshold"] = selected_threshold
```

```
powerbi_predictions["Predicted_Class"] = (
    test_scores >= selected_threshold
).astype(int)

powerbi_predictions["Prediction_Status"] = np.select(
    [
        powerbi_predictions["Predicted_Class"].eq(1)
        & powerbi_predictions["Actual_Class"].eq(1),

        powerbi_predictions["Predicted_Class"].eq(1)
        & powerbi_predictions["Actual_Class"].eq(0),

        powerbi_predictions["Predicted_Class"].eq(0)
        & powerbi_predictions["Actual_Class"].eq(1)
    ],
    [
        "True Positive",
        "False Positive",
        "False Negative"
    ],
    default="True Negative"
)

powerbi_predictions.to_csv(
    output_dir / "fraud_predictions_test.csv",
    index=False
)

# Summary metrics for the candidate threshold
tn, fp, fn, tp = confusion_matrix(
    y_test,
    (test_scores >= selected_threshold).astype(int),
    labels=[0, 1]
).ravel()

summary = pd.DataFrame([{
    "Model": "Logistic Regression",
    "Threshold": selected_threshold,
    "Test_Transactions": len(y_test),
    "Actual_Fraud": int(y_test.sum()),
    "True_Positives": int(tp),
    "False_Negatives": int(fn),
    "False_Positives": int(fp),
    "True_Negatives": int(tn),
    "Precision": tp / (tp + fp) if (tp + fp) else 0,
    "Recall": tp / (tp + fn) if (tp + fn) else 0,
    "Fraud_Alert_Count": int(tp + fp)
}])

summary.to_csv(
    output_dir / "model_summary.csv",
    index=False
)

# Save the threshold comparison from the test evaluation
test_threshold_comparison.to_csv(
    output_dir / "threshold_comparison_test.csv",
    index=False
)

print("Power BI files saved to:", output_dir.resolve())
print("\nFiles created:")
for file in output_dir.glob("*.csv"):
    print("-", file.name)
```

```
Power BI files saved to: /content/fraudshield_powerbi

Files created:
- model_summary.csv
- threshold_comparison_test.csv
- fraud_predictions_test.csv
```

```
# FraudShield AI - Transaction Amount Distribution Analysis

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
legitimate_amounts = df_clean.loc[
    df_clean["Class"] == 0, "Amount"
]

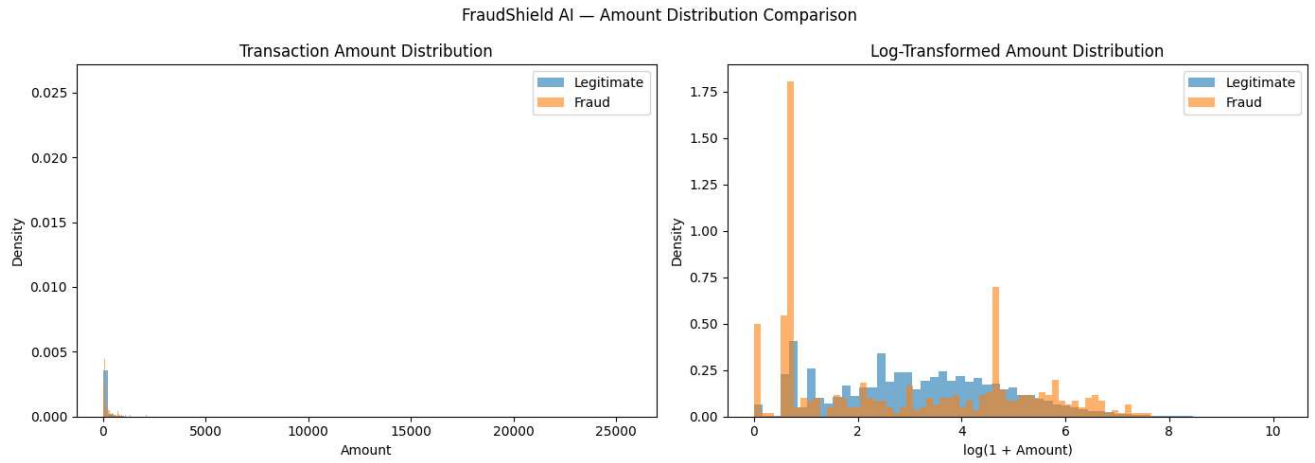
fraud_amounts = df_clean.loc[
    df_clean["Class"] == 1, "Amount"
]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Original amount distribution
axes[0].hist(
    legitimate_amounts,
    bins=100,
    density=True,
    alpha=0.6,
    label="Legitimate"
)
axes[0].hist(
    fraud_amounts,
    bins=100,
    density=True,
    alpha=0.6,
    label="Fraud"
)
axes[0].set_title("Transaction Amount Distribution")
axes[0].set_xlabel("Amount")
axes[0].set_ylabel("Density")
axes[0].legend()

# Plot 2: Log-transformed amount distribution
axes[1].hist(
    np.log1p(legitimate_amounts),
    bins=60,
    density=True,
    alpha=0.6,
    label="Legitimate"
)
axes[1].hist(
    np.log1p(fraud_amounts),
    bins=60,
    density=True,
    alpha=0.6,
    label="Fraud"
)
axes[1].set_title("Log-Transformed Amount Distribution")
axes[1].set_xlabel("log(1 + Amount)")
axes[1].set_ylabel("Density")
axes[1].legend()

plt.suptitle("FraudShield AI — Amount Distribution Comparison")
plt.tight_layout()
plt.show()
```




```
# FraudShield AI - Amount Distribution Summary

amount_distribution = (
    df_clean.groupby("Class")["Amount"]
    .agg(
        Transaction_Count="count",
        Mean="mean",
        Median="median",
        Standard_Deviation="std",
        Minimum="min",
        Maximum="max"
    )
)

amount_distribution.index = [
    "Legitimate" if i == 0 else "Fraud"
    for i in amount_distribution.index
]

display(amount_distribution.round(2))

amount_percentiles = (
    df_clean.groupby("Class")["Amount"]
    .quantile([0.10, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99])
    .unstack()
)

amount_percentiles.index = [
    "Legitimate" if i == 0 else "Fraud"
    for i in amount_percentiles.index
]

amount_percentiles.columns = [
    "P10", "P25", "P50", "P75", "P90", "P95", "P99"
]

display(amount_percentiles.round(2))
```

None Like what you see? Visit the data table notebook to learn more about interactive tables.

	P10	P25	P50	P75	P90	P95	P99
Legitimate	1.00	5.67	22.00	77.46	203.00	365.00	1018.06
Fraud	0.76	1.00	9.82	105.89	348.65	655.82	1364.14

```
Error: Runtime no longer has a reference to this dataframe, please re-run this cell and try again.
```

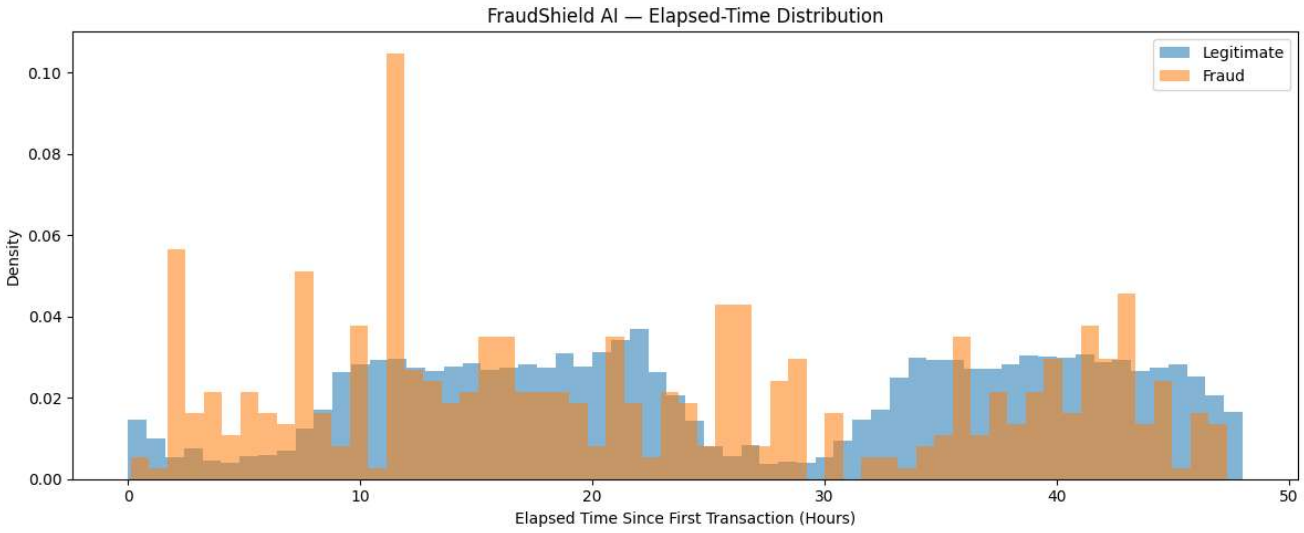
```
# FraudShield AI - Elapsed Time Distribution

plt.figure(figsize=(12, 5))
```

```
for class_value, label in [(0, "Legitimate"), (1, "Fraud")]:
    times_hours = (
        df_clean.loc[df_clean["Class"] == class_value, "Time"] / 3600
    )

    plt.hist(
        times_hours,
        bins=60,
        density=True,
        alpha=0.55,
        label=label
    )

plt.title("FraudShield AI — Elapsed-Time Distribution")
plt.xlabel("Elapsed Time Since First Transaction (Hours)")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()
```




```
# FraudShield AI - Overall Transaction Time Gaps

time_df = df_clean[["Time", "Class"]].copy()
time_df = time_df.sort_values("Time").reset_index(drop=True)

time_df["Time_Gap_Seconds"] = time_df["Time"].diff()

# Exclude the first row, which has no previous transaction
gap_df = time_df.dropna(subset=["Time_Gap_Seconds"]).copy()

gap_summary = (
    gap_df.groupby("Class")["Time_Gap_Seconds"]
    .agg(
        Transaction_Count="count",
        Mean_Gap_Seconds="mean",
        Median_Gap_Seconds="median",
        P25_Gap_Seconds=lambda s: s.quantile(0.25),
        P75_Gap_Seconds=lambda s: s.quantile(0.75)
    )
)

gap_summary.index = [
    "Legitimate" if i == 0 else "Fraud"
    for i in gap_summary.index
]

display(gap_summary.round(2))
```

	Transaction_Count	Mean_Gap_Seconds	Median_Gap_Seconds	P25_Gap_Seconds	P75_Gap_Seconds
Legitimate	283252	0.61	0.0	0.0	1.0
Fraud	473	1.15	1.0	0.0	1.0

```
# FraudShield AI - Time Gap Distribution

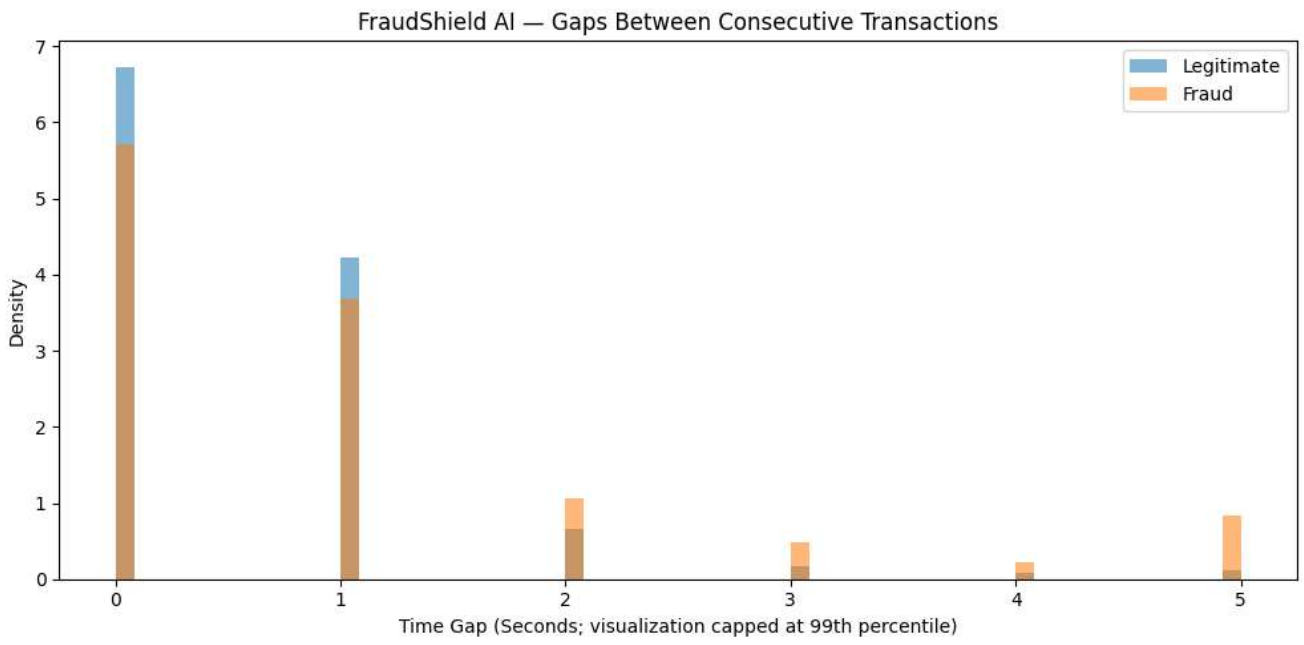
plt.figure(figsize=(10, 5))

for class_value, label in [(0, "Legitimate"), (1, "Fraud")]:
    gaps = gap_df.loc[
        gap_df["Class"] == class_value,
        "Time_Gap_Seconds"
    ]

    # Cap only the visualization at the 99th percentile
    # so very large gaps do not dominate the chart.
    upper_limit = gap_df["Time_Gap_Seconds"].quantile(0.99)
    gaps = gaps.clip(upper=upper_limit)

    plt.hist(
        gaps,
        bins=60,
        density=True,
        alpha=0.55,
        label=label
    )

plt.title("FraudShield AI — Gaps Between Consecutive Transactions")
plt.xlabel("Time Gap (Seconds; visualization capped at 99th percentile)")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()
```




```
# Step 2.1 — Reproducible Model Development Split

from sklearn.model_selection import train_test_split

X_fit, X_val, y_fit, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.25,
    stratify=y_train,
    random_state=42
)

print("MODEL DEVELOPMENT SPLIT")
```

```
print("-" * 40)
print("Fit rows:", len(X_fit))
print("Fit fraud cases:", int(y_fit.sum()))
print("Validation rows:", len(X_val))
print("Validation fraud cases:", int(y_val.sum()))
print("Existing test rows:", len(X_test))
print("Existing test fraud cases:", int(y_test.sum()))

assert X_fit.index.intersection(X_val.index).empty
assert X_fit.index.intersection(X_test.index).empty
assert X_val.index.intersection(X_test.index).empty
```

```
MODEL DEVELOPMENT SPLIT
----------------------------------------
Fit rows: 170235
Fit fraud cases: 284
Validation rows: 56745
Validation fraud cases: 94
Existing test rows: 56746
Existing test fraud cases: 95
```

```
# Step 2.2 — Train and Compare Models

import pandas as pd
import numpy as np

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    average_precision_score,
    roc_auc_score,
    confusion_matrix
)

models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(
            class_weight="balanced",
            max_iter=2000,
            random_state=42
        ))
    ]),

    "Random Forest": RandomForestClassifier(
        n_estimators=300,
        max_depth=None,
        min_samples_leaf=2,
        class_weight="balanced_subsample",
        random_state=42,
        n_jobs=-1
    )
}

fitted_models = {}
validation_scores = {}
comparison_rows = []

for model_name, model in models.items():
    print(f"Training {model_name}...")

    model.fit(X_fit, y_fit)
    fitted_models[model_name] = model

    scores = model.predict_proba(X_val)[:, 1]
    validation_scores[model_name] = scores

    predictions = (scores >= 0.5).astype(int)

    tn, fp, fn, tp = confusion_matrix(
        y_val, predictions, labels=[0, 1]
    ).ravel()
```

```
    comparison_rows.append({
        "Model": model_name,
        "Precision": precision_score(
            y_val, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_val, predictions, zero_division=0
        ),
        "F1_Score": f1_score(
            y_val, predictions, zero_division=0
        ),
        "Average_Precision": average_precision_score(
            y_val, scores
        ),
        "ROC_AUC": roc_auc_score(y_val, scores),
        "False_Alerts": int(fp),
        "Fraud_Missed": int(fn),
        "Fraud_Caught": int(tp)
    })

model_comparison = pd.DataFrame(comparison_rows)

display(
    model_comparison
    .sort_values("Average_Precision", ascending=False)
    .round(4)
)
```

```
Training Logistic Regression...
Training Random Forest...
```

	Model	Precision	Recall	F1_Score	Average_Precision	ROC_AUC	False_Alerts	Fraud_Missed	Fraud_Caught
1	Random Forest	0.9186	0.8404	0.8778	0.8757	0.960	7	15	79
0	Logistic Regression	0.0538	0.9043	0.1016	0.7876	0.979	1495	9	85

```
# Step 2.3 — Validation Threshold Analysis

thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

threshold_rows = []

for model_name, scores in validation_scores.items():
    for threshold in thresholds:
        predictions = (scores >= threshold).astype(int)

        tn, fp, fn, tp = confusion_matrix(
            y_val, predictions, labels=[0, 1]
        ).ravel()

        threshold_rows.append({
            "Model": model_name,
            "Threshold": threshold,
            "Fraud_Caught": int(tp),
            "Fraud_Missed": int(fn),
            "False_Alerts": int(fp),
            "Total_Alerts": int(tp + fp),
            "Precision_Pct": precision_score(
                y_val, predictions, zero_division=0
            ) * 100,
            "Recall_Pct": recall_score(
                y_val, predictions, zero_division=0
            ) * 100,
            "F1_Score": f1_score(
                y_val, predictions, zero_division=0
            )
        })

threshold_comparison = pd.DataFrame(threshold_rows)

display(
    threshold_comparison.round(3)
    .sort_values(["Model", "Threshold"])
    .reset_index(drop=True)
)
```

	Model	Threshold	Fraud_Caught	Fraud_Missed	False_Alerts	Total_Alerts	Precision_Pct	Recall_Pct	F1_Score
0	Logistic Regression	0.1	90	4	9348	9438	0.954	95.745	0.019
1	Logistic Regression	0.2	88	6	4818	4906	1.794	93.617	0.035
2	Logistic Regression	0.3	86	8	3070	3156	2.725	91.489	0.053
3	Logistic Regression	0.4	86	8	2095	2181	3.943	91.489	0.076
4	Logistic Regression	0.5	85	9	1495	1580	5.380	90.426	0.102
5	Logistic Regression	0.6	85	9	1021	1106	7.685	90.426	0.142
6	Logistic Regression	0.7	85	9	743	828	10.266	90.426	0.184
7	Logistic Regression	0.8	84	10	494	578	14.533	89.362	0.250
8	Logistic Regression	0.9	83	11	254	337	24.629	88.298	0.385
9	Random Forest	0.1	83	11	20	103	80.583	88.298	0.843
10	Random Forest	0.2	81	13	11	92	88.043	86.170	0.871
11	Random Forest	0.3	81	13	9	90	90.000	86.170	0.880
12	Random Forest	0.4	79	15	8	87	90.805	84.043	0.873

```
# Step 2.4 — Example Alert Capacity Filter
# Change 500 to your actual review-capacity target if known.

max_false_alerts = 500

eligible_thresholds = threshold_comparison[
    threshold_comparison["False_Alerts"] <= max_false_alerts
].copy()

if eligible_thresholds.empty:
    print("No threshold meets this alert-capacity target.")
else:
    display(
        eligible_thresholds.sort_values(
            ["Fraud_Caught", "Precision_Pct"],
            ascending=False
        ).round(3)
    )
```

	Model	Threshold	Fraud_Caught	Fraud_Missed	False_Alerts	Total_Alerts	Precision_Pct	Recall_Pct	F1_Score
7	Logistic Regression	0.8	84	10	494	578	14.533	89.362	0.250
9	Random Forest	0.1	83	11	20	103	80.583	88.298	0.843
8	Logistic Regression	0.9	83	11	254	337	24.629	88.298	0.385
11	Random Forest	0.3	81	13	9	90	90.000	86.170	0.880
10	Random Forest	0.2	81	13	11	92	88.043	86.170	0.871
13	Random Forest	0.5	79	15	7	86	91.860	84.043	0.878
12	Random Forest	0.4	79	15	8	87	90.805	84.043	0.873
14	Random Forest	0.6	74	20	6	80	92.500	78.723	0.851
15	Random Forest	0.7	71	23	6	77	92.208	75.532	0.830
16	Random Forest	0.8	60	34	2	62	96.774	63.830	0.769

```
import pandas as pd
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    average_precision_score,
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score
)

# 1. Train the selected model on the full training split
final_rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_leaf=2,
    class_weight="balanced_subsample",
    random_state=42,
    n_jobs=-1
)

final_rf.fit(X_train, y_train)

# 2. Generate test-set fraud probabilities
test_probabilities = final_rf.predict_proba(X_test)[:, 1]

# 3. Apply the threshold selected using validation data
selected_threshold = 0.3
test_predictions = (
    test_probabilities >= selected_threshold
).astype(int)

# 4. Calculate evaluation metrics
tn, fp, fn, tp = confusion_matrix(
    y_test, test_predictions, labels=[0, 1]
).ravel()

print("FINAL RANDOM FOREST EVALUATION")
print("-" * 45)
print(f"Threshold: {selected_threshold}")
print(f"Test transactions: {len(y_test):,}")
print(f"Actual fraud cases: {int(y_test.sum()):,}")
print(f"Fraud caught (TP): {tp:,}")
print(f"Fraud missed (FN): {fn:,}")
print(f"False alerts (FP): {fp:,}")
print(f"Correct legitimate transactions (TN): {tn:,}")
print(f"Total alerts: {tp + fp:,}")
print(f"Precision: {precision_score(y_test, test_predictions, zero_division=0):.4f}")
print(f"Recall: {recall_score(y_test, test_predictions, zero_division=0):.4f}")
print(f"F1-score: {f1_score(y_test, test_predictions, zero_division=0):.4f}")
print(f"Average Precision: {average_precision_score(y_test, test_probabilities):.4f}")
print(f"ROC-AUC: {roc_auc_score(y_test, test_probabilities):.4f}")

print("\nClassification Report")
print(classification_report(
    y_test, test_predictions,
    target_names=["Legitimate", "Fraud"],
    zero_division=0
))
```

```
FINAL RANDOM FOREST EVALUATION
---------------------------------------------
Threshold: 0.3
Test transactions: 56,746
Actual fraud cases: 95
Fraud caught (TP): 71
Fraud missed (FN): 24
False alerts (FP): 6
Correct legitimate transactions (TN): 56,645
Total alerts: 77
Precision: 0.9221
Recall: 0.7474
F1-score: 0.8256
Average Precision: 0.8109
ROC-AUC: 0.9427
```

```
Classification Report
              precision    recall  f1-score   support

  Legitimate       1.00      1.00      1.00     56651
       Fraud       0.92      0.75      0.83        95

    accuracy                           1.00     56746
   macro avg       0.96      0.87      0.91     56746
weighted avg       1.00      1.00      1.00     56746
```

```
feature_importance = pd.DataFrame({
    "Feature": X_train.columns,
    "Importance": final_rf.feature_importances_
}).sort_values("Importance", ascending=False)

display(feature_importance.head(15))

feature_importance.to_csv(
    "fraudshield_feature_importance.csv",
    index=False
)
```

	Feature	Importance
14	V14	0.171147
10	V10	0.121662
4	V4	0.108970
12	V12	0.108339
17	V17	0.088004
11	V11	0.059315
16	V16	0.057041
3	V3	0.052788
7	V7	0.027823
2	V2	0.027631
9	V9	0.019504
21	V21	0.016479
27	V27	0.013086
19	V19	0.012433
5	V5	0.010694

```
import pandas as pd
import joblib
from pathlib import Path
from sklearn.metrics import confusion_matrix

# Create an output folder
output_dir = Path("fraudshield_outputs")
output_dir.mkdir(exist_ok=True)

# 1. Export test-split predictions for Power BI
powerbi_predictions = X_test.copy()

powerbi_predictions.insert(
    0, "Dataset_Row_Index", X_test.index
)
powerbi_predictions["Actual_Class"] = y_test
powerbi_predictions["Fraud_Probability"] = test_probabilities
powerbi_predictions["Predicted_Class"] = test_predictions
powerbi_predictions["Alert_Status"] = (
    powerbi_predictions["Predicted_Class"]
    .map({0: "No Alert", 1: "Fraud Alert"})
)
powerbi_predictions["Model_Threshold"] = selected_threshold
```

```
powerbi_predictions.to_csv(
    output_dir / "fraudshield_test_predictions.csv",
    index=False
)

# 2. Export feature importance
feature_importance.to_csv(
    output_dir / "fraudshield_feature_importance.csv",
    index=False
)

# 3. Export a model performance summary
tn, fp, fn, tp = confusion_matrix(
    y_test, test_predictions, labels=[0, 1]
).ravel()

performance_summary = pd.DataFrame([{
    "Model": "Random Forest",
    "Threshold": selected_threshold,
    "Test_Transactions": len(y_test),
    "Actual_Fraud": int(y_test.sum()),
    "Fraud_Caught": int(tp),
    "Fraud_Missed": int(fn),
    "False_Alerts": int(fp),
    "Correct_Legitimate": int(tn),
    "Total_Alerts": int(tp + fp),
    "Precision": precision_score(
        y_test, test_predictions, zero_division=0
    ),
    "Recall": recall_score(
        y_test, test_predictions, zero_division=0
    ),
    "F1_Score": f1_score(
        y_test, test_predictions, zero_division=0
    ),
    "Average_Precision": average_precision_score(
        y_test, test_probabilities
    ),
    "ROC_AUC": roc_auc_score(y_test, test_probabilities)
}])

performance_summary.to_csv(
    output_dir / "fraudshield_model_performance.csv",
    index=False
)

# 4. Save the trained model for reuse
joblib.dump(
    final_rf,
    output_dir / "fraudshield_random_forest.joblib"
)

print("Export complete. Files created:")
for file_path in sorted(output_dir.iterdir()):
    print(file_path)
```

```
Export complete. Files created:
fraudshield_outputs/fraudshield_feature_importance.csv
fraudshield_outputs/fraudshield_model_performance.csv
fraudshield_outputs/fraudshield_random_forest.joblib
fraudshield_outputs/fraudshield_test_predictions.csv
```

```
import pandas as pd
import numpy as np
from pathlib import Path

# Reproducible synthetic scenario data
rng = np.random.default_rng(42)
N = 5000

output_dir = Path("fraudshield_outputs")
output_dir.mkdir(exist_ok=True)

# Simulated customer and merchant dimensions
```

```
cities = [
    "Pune", "Mumbai", "Bengaluru", "Hyderabad",
    "Delhi", "Chennai", "Kolkata", "Ahmedabad"
]

merchant_categories = [
    "Grocery", "Electronics", "Travel", "Dining",
    "Fuel", "Online Retail", "Entertainment", "Pharmacy"
]

customer_ids = [f"SIM_CUST_{i:05d}" for i in range(1, 1001)]
merchant_ids = [f"SIM_MERCH_{i:04d}" for i in range(1, 101)]

# Simulated customer home locations
customer_home_city = {
    customer: rng.choice(cities)
    for customer in customer_ids
}

# Generate synthetic transactions across 30 days
timestamps = (
    pd.Timestamp("2026-01-01")
    + pd.to_timedelta(
        rng.integers(0, 30 * 24 * 60 * 60, size=N),
        unit="s"
    )
)

scenario_df = pd.DataFrame({
    "SIMULATED_Transaction_ID": [
        f"SIM_TXN_{i:06d}" for i in range(1, N + 1)
    ],
    "SIMULATED_Timestamp": timestamps,
    "SIMULATED_Customer_ID": rng.choice(customer_ids, size=N),
    "SIMULATED_Merchant_ID": rng.choice(merchant_ids, size=N),
    "SIMULATED_Merchant_Category": rng.choice(
        merchant_categories, size=N
    ),
    "SIMULATED_Transaction_Amount": np.round(
        rng.lognormal(mean=3.5, sigma=1.1, size=N), 2
    ),
    "SIMULATED_Transaction_City": rng.choice(cities, size=N)
})

scenario_df["SIMULATED_Customer_Home_City"] = (
    scenario_df["SIMULATED_Customer_ID"]
    .map(customer_home_city)
)

# Illustrative geographic mismatch:
# transaction city differs from the simulated home city.
scenario_df["SIMULATED_Geographic_Anomaly"] = (
    scenario_df["SIMULATED_Transaction_City"]
    != scenario_df["SIMULATED_Customer_Home_City"]
)

# Calculate how many transactions the same customer made
# during the preceding 60 minutes, excluding the current one.
scenario_df = scenario_df.sort_values(
    ["SIMULATED_Customer_ID", "SIMULATED_Timestamp"]
).reset_index(drop=True)

scenario_df["SIMULATED_Transactions_Previous_60_Min"] = 0

for customer_id, group in scenario_df.groupby(
    "SIMULATED_Customer_ID", sort=False
):
    positions = group.index.to_numpy()
    times = (
        group["SIMULATED_Timestamp"]
        .astype("int64")
        .to_numpy()
    )

    window_ns = 60 * 60 * 1_000_000_000

    # Left boundary is inclusive; current transaction excluded.
```

```
        counts = np.zeros(len(times), dtype=int)

        for i in range(len(times)):
            left = np.searchsorted(
                times, times[i] - window_ns, side="left"
            )
            counts[i] = i - left

        scenario_df.loc[
            positions, "SIMULATED_Transactions_Previous_60_Min"
        ] = counts

    # Illustrative rule-based alert flags.
    # These are designed scenarios, NOT verified fraud labels.
    scenario_df["SIMULATED_Velocity_Anomaly"] = (
        scenario_df["SIMULATED_Transactions_Previous_60_Min"] >= 3
    )

    scenario_df["SIMULATED_High_Amount_Flag"] = (
        scenario_df["SIMULATED_Transaction_Amount"] >= 1000
    )

    scenario_df["SIMULATED_Rule_Alert"] = (
        scenario_df["SIMULATED_Geographic_Anomaly"]
        | scenario_df["SIMULATED_Velocity_Anomaly"]
        | scenario_df["SIMULATED_High_Amount_Flag"]
    )

    # Explicit data provenance and limitation labels
    scenario_df["SIMULATED_Data_Source"] = "SIMULATED"
    scenario_df["SIMULATED_Alert_Basis"] = (
        "Illustrative rules only; not a confirmed fraud outcome"
    )

    # Export
    scenario_path = (
        output_dir / "fraudshield_simulated_business_scenarios.csv"
    )
    scenario_df.to_csv(scenario_path, index=False)

    print("SIMULATED SCENARIO DATASET CREATED")
    print("-" * 45)
    print(f"Rows: {len(scenario_df):,}")
    print(f"Unique simulated customers: "
          f"{scenario_df['SIMULATED_Customer_ID'].nunique():,}")
    print(f"Unique simulated merchants: "
          f"{scenario_df['SIMULATED_Merchant_ID'].nunique():,}")
    print(f"Simulated rule alerts: "
          f"{scenario_df['SIMULATED_Rule_Alert'].sum():,}")
    print(f"Saved to: {scenario_path}")
    display(scenario_df.head())
```

```
SIMULATED SCENARIO DATASET CREATED
---------------------------------------------
Rows: 5,000
Unique simulated customers: 995
Unique simulated merchants: 100
Simulated rule alerts: 4,360
Saved to: fraudshield_outputs/fraudshield_simulated_business_scenarios.csv
```

	SIMULATED_Transaction_ID	SIMULATED_Timestamp	SIMULATED_Customer_ID	SIMULATED_Merchant_ID	SIMULATED_Merchant_Category	SIM
0	SIM_TXN_001006	2026-01-04 20:54:05	SIM_CUST_00001	SIM_MERCH_0057	Electronics	
1	SIM_TXN_000906	2026-01-08 08:15:12	SIM_CUST_00001	SIM_MERCH_0046	Electronics	
2	SIM_TXN_004964	2026-01-09 08:18:58	SIM_CUST_00001	SIM_MERCH_0090	Grocery	
3	SIM_TXN_000302	2026-01-13 00:58:33	SIM_CUST_00001	SIM_MERCH_0089	Fuel	
4	SIM_TXN_003284	2026-01-14 00:43:53	SIM_CUST_00001	SIM_MERCH_0072	Electronics	

```
# All outputs in this section describe simulated scenarios only.

simulated_summary = pd.DataFrame([
    {
        "SIMULATED_Metric": "Total Transactions",
        "SIMULATED_Value": len(scenario_df)
    },
    {
        "SIMULATED_Metric": "Geographic Anomaly Flags",
        "SIMULATED_Value": int(
            scenario_df["SIMULATED_Geographic_Anomaly"].sum()
        )
    },
    {
        "SIMULATED_Metric": "Velocity Anomaly Flags",
        "SIMULATED_Value": int(
            scenario_df["SIMULATED_Velocity_Anomaly"].sum()
        )
    },
    {
        "SIMULATED_Metric": "High Amount Flags",
        "SIMULATED_Value": int(
            scenario_df["SIMULATED_High_Amount_Flag"].sum()
        )
    },
    {
        "SIMULATED_Metric": "Rule Alerts",
        "SIMULATED_Value": int(
            scenario_df["SIMULATED_Rule_Alert"].sum()
        )
    }
])

display(simulated_summary)

simulated_summary.to_csv(
    output_dir / "fraudshield_simulated_scenario_summary.csv",
    index=False
)

# Simulated alert counts by merchant category
category_summary = (
    scenario_df.groupby("SIMULATED_Merchant_Category")
    .agg(
        SIMULATED_Transactions=(
            "SIMULATED_Transaction_ID", "count"
        ),
        SIMULATED_Rule_Alerts=(
            "SIMULATED_Rule_Alert", "sum"
        ),
        SIMULATED_Average_Amount=(
            "SIMULATED_Transaction_Amount", "mean"
        )
    )
    .reset_index()
)

category_summary["SIMULATED_Average_Amount"] = (
    category_summary["SIMULATED_Average_Amount"].round(2)
)

display(category_summary)

category_summary.to_csv(
    output_dir / "fraudshield_simulated_category_summary.csv",
    index=False
)
```

	SIMULATED_Metric	SIMULATED_Value
0	Total Transactions	5000
1	Geographic Anomaly Flags	4360
2	Velocity Anomaly Flags	0
3	High Amount Flags	4
4	Rule Alerts	4360

	SIMULATED_Merchant_Category	SIMULATED_Transactions	SIMULATED_Rule_Alerts	SIMULATED_Average_Amount
0	Dining	622	548	69.20
1	Electronics	656	575	55.30
2	Entertainment	602	537	61.38
3	Fuel	617	531	60.35
4	Grocery	641	550	58.73
5	Online Retail	650	574	62.18
6	Pharmacy	607	519	59.12
7	Travel	605	526	61.11

Next steps: Generate code with simulated_summary | New interactive sheet | Generate code with category_summary | New interactive sheet

```
import pandas as pd
import numpy as np
from pathlib import Path
from sklearn.metrics import (
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
    average_precision_score,
    roc_auc_score
)

# ============================================================
# PHASE 4 — POWER BI EXPORTS
# REAL DATA AND SIMULATED DATA ARE KEPT SEPARATE
# ============================================================

out = Path("fraudshield_outputs")
out.mkdir(parents=True, exist_ok=True)

# ------------------------------------------------------------
# 1. REAL EDA: fraud class summary
# ------------------------------------------------------------
class_summary = (
    df_clean.groupby("Class")
    .agg(Transaction_Count=("Class", "size"))
    .reset_index()
)

class_summary["Class_Label"] = class_summary["Class"].map({
    0: "Legitimate",
    1: "Fraud"
})
class_summary["Transaction_Share_Pct"] = (
    class_summary["Transaction_Count"]
    / len(df_clean) * 100
).round(4)
class_summary["Data_Source"] = "REAL_KAGGLE"

class_summary.to_csv(
    out / "real_eda_class_summary.csv", index=False
)

# ------------------------------------------------------------
# 2. REAL EDA: amount-range fraud rates
# ------------------------------------------------------------
amount_bins = [0, 10, 50, 100, 250, 500, 1000, np.inf]
```

```
amount_labels = [
    "0-10", "10-50", "50-100", "100-250",
    "250-500", "500-1000", "1000+"
]

amount_data = df_clean.copy()
amount_data["Amount_Range"] = pd.cut(
    amount_data["Amount"],
    bins=amount_bins,
    labels=amount_labels,
    right=False,
    include_lowest=True
)

amount_summary = (
    amount_data.groupby("Amount_Range", observed=False)
    .agg(
        Transaction_Count=("Class", "size"),
        Fraud_Count=("Class", "sum"),
        Average_Amount=("Amount", "mean"),
        Median_Amount=("Amount", "median")
    )
    .reset_index()
)

amount_summary["Legitimate_Count"] = (
    amount_summary["Transaction_Count"]
    - amount_summary["Fraud_Count"]
)
amount_summary["Fraud_Rate_Pct"] = (
    amount_summary["Fraud_Count"]
    / amount_summary["Transaction_Count"].replace(0, np.nan)
    * 100
).round(4)
amount_summary["Share_of_All_Fraud_Pct"] = (
    amount_summary["Fraud_Count"]
    / df_clean["Class"].sum() * 100
).round(4)
amount_summary["Average_Amount"] = (
    amount_summary["Average_Amount"].round(2)
)
amount_summary["Median_Amount"] = (
    amount_summary["Median_Amount"].round(2)
)
amount_summary["Data_Source"] = "REAL_KAGGLE"

amount_summary.to_csv(
    out / "real_eda_amount_range_summary.csv", index=False
)

# ------------------------------------------------------------
# 3. REAL EDA: elapsed-time intervals
# Time is elapsed seconds, NOT wall-clock time.
# ------------------------------------------------------------
time_data = df_clean.copy()
time_data["Elapsed_Hours"] = time_data["Time"] / 3600

time_data["Elapsed_Interval"] = pd.cut(
    time_data["Elapsed_Hours"],
    bins=np.arange(0, 54, 6),
    right=False,
    labels=[
        f"{i}-{i+6} hours"
        for i in range(0, 48, 6)
    ]
)

elapsed_summary = (
    time_data.groupby("Elapsed_Interval", observed=False)
    .agg(
        Transaction_Count=("Class", "size"),
        Fraud_Count=("Class", "sum")
    )
    .reset_index()
)

elapsed_summary["Fraud_Rate_Pct"] = (
```

```
    elapsed_summary["Fraud_Count"]
    / elapsed_summary["Transaction_Count"].replace(0, np.nan)
    * 100
).round(4)
elapsed_summary["Share_of_All_Fraud_Pct"] = (
    elapsed_summary["Fraud_Count"]
    / df_clean["Class"].sum() * 100
).round(4)
elapsed_summary["Data_Source"] = "REAL_KAGGLE"
elapsed_summary["Time_Interpretation"] = (
    "Elapsed hours from dataset start; not time of day"
)

elapsed_summary.to_csv(
    out / "real_eda_elapsed_time_summary.csv", index=False
)

# ------------------------------------------------------------
# 4. REAL EDA: gaps between consecutive transactions
# ------------------------------------------------------------
gap_data = df_clean.sort_values("Time").copy()
gap_data["Gap_Seconds"] = gap_data["Time"].diff()
gap_data["Gap_Minutes"] = gap_data["Gap_Seconds"] / 60

gap_data["Gap_Range"] = pd.cut(
    gap_data["Gap_Seconds"],
    bins=[-np.inf, 0, 1, 10, 60, 300, 1800, np.inf],
    labels=[
        "Zero_or_negative", "0-1 sec", "1-10 sec",
        "10-60 sec", "1-5 min", "5-30 min", "30+ min"
    ],
    right=True
)

gap_summary = (
    gap_data.dropna(subset=["Gap_Range"])
    .groupby(["Gap_Range", "Class"], observed=False)
    .agg(Transaction_Count=("Class", "size"))
    .reset_index()
)

gap_summary["Class_Label"] = gap_summary["Class"].map({
    0: "Legitimate",
    1: "Fraud"
})
gap_summary["Data_Source"] = "REAL_KAGGLE"
gap_summary["Gap_Interpretation"] = (
    "Difference in elapsed Time between adjacent dataset rows; "
    "not a customer-level velocity measure"
)

gap_summary.to_csv(
    out / "real_eda_time_gap_summary.csv", index=False
)

# Export a row-level gap file for optional Power BI analysis
gap_data[[
    "Time", "Class", "Gap_Seconds", "Gap_Minutes", "Gap_Range"
]].to_csv(
    out / "real_eda_transaction_time_gaps.csv", index=False
)

# ------------------------------------------------------------
# 5. REAL MODEL: test predictions
# ------------------------------------------------------------
test_export = X_test.copy()
test_export.insert(0, "Dataset_Row_Index", X_test.index)
test_export["Actual_Class"] = np.asarray(y_test)
test_export["Fraud_Probability"] = test_probabilities
test_export["Predicted_Class"] = test_predictions
test_export["Alert_Status"] = np.where(
    test_export["Predicted_Class"] == 1,
    "Fraud Alert",
    "No Alert"
)
test_export["Model"] = "Random Forest"
test_export["Threshold"] = selected_threshold
```

```
test_export["Data_Source"] = "REAL_KAGGLE_TEST_EVALUATION"

test_export.to_csv(
    out / "real_model_test_predictions.csv", index=False
)

# ------------------------------------------------------------
# 6. REAL MODEL: performance and confusion matrix
# ------------------------------------------------------------
tn, fp, fn, tp = confusion_matrix(
    y_test, test_predictions, labels=[0, 1]
).ravel()

model_performance = pd.DataFrame([{
    "Model": "Random Forest",
    "Threshold": selected_threshold,
    "Test_Transactions": len(y_test),
    "Actual_Fraud": int(np.sum(y_test)),
    "Fraud_Caught_TP": int(tp),
    "Fraud_Missed_FN": int(fn),
    "False_Alerts_FP": int(fp),
    "Correct_Legitimate_TN": int(tn),
    "Total_Alerts": int(tp + fp),
    "Precision": precision_score(
        y_test, test_predictions, zero_division=0
    ),
    "Recall": recall_score(
        y_test, test_predictions, zero_division=0
    ),
    "F1_Score": f1_score(
        y_test, test_predictions, zero_division=0
    ),
    "Average_Precision": average_precision_score(
        y_test, test_probabilities
    ),
    "ROC_AUC": roc_auc_score(y_test, test_probabilities),
    "Data_Source": "REAL_KAGGLE_TEST_EVALUATION"
}])

model_performance.to_csv(
    out / "real_model_performance.csv", index=False
)

confusion_export = pd.DataFrame([
    {
        "Actual_Class": "Legitimate",
        "Predicted_Class": "Legitimate",
        "Count": int(tn),
        "Metric": "True Negative"
    },
    {
        "Actual_Class": "Legitimate",
        "Predicted_Class": "Fraud",
        "Count": int(fp),
        "Metric": "False Positive"
    },
    {
        "Actual_Class": "Fraud",
        "Predicted_Class": "Legitimate",
        "Count": int(fn),
        "Metric": "False Negative"
    },
    {
        "Actual_Class": "Fraud",
        "Predicted_Class": "Fraud",
        "Count": int(tp),
        "Metric": "True Positive"
    }
])
confusion_export["Data_Source"] = "REAL_KAGGLE_TEST_EVALUATION"

confusion_export.to_csv(
    out / "real_confusion_matrix.csv", index=False
)

# ------------------------------------------------------------
# 7. Feature importance
```

```
# ------------------------------------------------------------
feature_importance.to_csv(
    out / "real_model_feature_importance.csv", index=False
)

# ------------------------------------------------------------
# 8. Keep existing simulated exports separate
# ------------------------------------------------------------
simulated_files = {
    "fraudshield_simulated_business_scenarios.csv":
        "SIMULATED_business_scenarios.csv",
    "fraudshield_simulated_scenario_summary.csv":
        "SIMULATED_scenario_summary.csv",
    "fraudshield_simulated_category_summary.csv":
        "SIMULATED_category_summary.csv"
}

for source_name, target_name in simulated_files.items():
    source_path = out / source_name
    if source_path.exists():
        data = pd.read_csv(source_path)
        data.to_csv(out / target_name, index=False)
        print(f"Preserved simulated output: {target_name}")
    else:
        print(f"Not found yet (create in Phase 3): {source_name}")

print("\nCORE REAL-DATA EXPORTS CREATED")
print("-" * 45)
for file_path in sorted(out.glob("real_*.csv")):
    print(f"{file_path.name}: {file_path.stat().st_size:,} bytes")
```

```
Preserved simulated output: SIMULATED_business_scenarios.csv
Preserved simulated output: SIMULATED_scenario_summary.csv
Preserved simulated output: SIMULATED_category_summary.csv

CORE REAL-DATA EXPORTS CREATED
---------------------------------------------
real_confusion_matrix.csv: 306 bytes
real_eda_amount_range_summary.csv: 564 bytes
real_eda_class_summary.csv: 141 bytes
real_eda_elapsed_time_summary.csv: 896 bytes
real_eda_time_gap_summary.csv: 1,923 bytes
real_eda_transaction_time_gaps.csv: 10,972,889 bytes
real_model_feature_importance.csv: 753 bytes
real_model_performance.csv: 360 bytes
real_model_test_predictions.csv: 33,951,454 bytes
```

```
# Check available variables
print([name for name in globals() if "threshold" in name.lower()])
```

```
['thresholds', 'threshold', 'threshold_results', 'threshold_results_test', 'test_threshold_comparison', 'selected_threshold', 't
```

```
import pandas as pd
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix
from pathlib import Path

thresholds = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]

# Retrieve the fitted models from the fitted_models dictionary created in cell irS2tgv14qcA
models = {
    "Logistic Regression": fitted_models["Logistic Regression"],
    "Random Forest": fitted_models["Random Forest"]
}

rows = []

for model_name, model in models.items():
    probabilities = model.predict_proba(X_val)[:, 1]

    for threshold in thresholds:
        predictions = (probabilities >= threshold).astype(int)
        tn, fp, fn, tp = confusion_matrix(
            y_val, predictions, labels=[0, 1]
        ).ravel()

        rows.append({
```

```
            "Model": model_name,
            "Threshold": threshold,
            "TP": int(tp),
            "FN": int(fn),
            "FP": int(fp),
            "TN": int(tn),
            "Alerts": int(tp + fp),
            "Precision_Pct": precision_score(
                y_val, predictions, zero_division=0
            ) * 100,
            "Recall_Pct": recall_score(
                y_val, predictions, zero_division=0
            ) * 100,
            "F1_Score": f1_score(
                y_val, predictions, zero_division=0
            ),
            "Data_Source": "REAL_KAGGLE_VALIDATION",
            "Evaluation_Split": "Validation"
        })

threshold_results = pd.DataFrame(rows)

output_dir = Path("fraudshield_outputs")
output_dir.mkdir(parents=True, exist_ok=True)

threshold_results.to_csv(
    output_dir / "real_threshold_comparison.csv",
    index=False
)

display(threshold_results)
print("Saved successfully.")
```

	Model	Threshold	TP	FN	FP	TN	Alerts	Precision_Pct	Recall_Pct	F1_Score	Data_Source	Evaluation
0	Logistic Regression	0.1	90	4	9348	47303	9438	0.953592	95.744681	0.018884	REAL_KAGGLE_VALIDATION	Va
1	Logistic Regression	0.2	88	6	4818	51833	4906	1.793722	93.617021	0.035200	REAL_KAGGLE_VALIDATION	Va
2	Logistic Regression	0.3	86	8	3070	53581	3156	2.724968	91.489362	0.052923	REAL_KAGGLE_VALIDATION	Va
3	Logistic Regression	0.4	86	8	2095	54556	2181	3.943145	91.489362	0.075604	REAL_KAGGLE_VALIDATION	Va
4	Logistic Regression	0.5	85	9	1495	55156	1580	5.379747	90.425532	0.101553	REAL_KAGGLE_VALIDATION	Va
5	Logistic Regression	0.6	85	9	1021	55630	1106	7.685353	90.425532	0.141667	REAL_KAGGLE_VALIDATION	Va
6	Logistic Regression	0.7	85	9	743	55908	828	10.265700	90.425532	0.184382	REAL_KAGGLE_VALIDATION	Va
7	Logistic Regression	0.8	84	10	494	56157	578	14.532872	89.361702	0.250000	REAL_KAGGLE_VALIDATION	Va
8	Logistic Regression	0.9	83	11	254	56397	337	24.629080	88.297872	0.385151	REAL_KAGGLE_VALIDATION	Va
9	Random Forest	0.1	83	11	20	56631	103	80.582524	88.297872	0.842640	REAL_KAGGLE_VALIDATION	Va
10	Random Forest	0.2	81	13	11	56640	92	88.043478	86.170213	0.870968	REAL_KAGGLE_VALIDATION	Va
11	Random Forest	0.3	81	13	9	56642	90	90.000000	86.170213	0.880435	REAL_KAGGLE_VALIDATION	Va

Next steps: Generate code with threshold_results | New interactive sheet

```
# FraudShield AI — Project Summary & Interview Notes

## 1. Business Problem
Credit card fraud detection is an imbalanced classification problem. Fraudulent transactions are rare, so a model can achieve m

## 2. Dataset
Source: Kaggle Credit Card Fraud Detection dataset.
```

```
Original rows: 284,807
Cleaned rows: 283,726
Duplicate rows removed: 1,081
```